

Project Report : CS 7643

Indoor Localization from Wi-Fi Fingerprints: A Robustness Study Under Data Perturbations

Jason W Park Wu, Di Kiyoshi Watanabe
Georgia Institute of Technology
{jasonpark9001, dwu400, kw53}@gatech.edu

Abstract

Indoor localization using Wi-Fi fingerprints is widely studied, but real-world performance can degrade when RSSI measurements are unstable, incomplete, or shifted across devices. In this project, we evaluate five deep learning model families—MLP, CNN, LSTM, standard Transformer, and Set Transformer—on the UJIIndoorLoc dataset under a shared preprocessing, training, evaluation, robustness-testing, and latency-benchmarking framework. We measure clean-test performance, retained performance under access point (AP) dropout, received signal strength indicator (RSSI) bias, Gaussian noise, combined perturbations, and batch-1 CPU inference latency.

Our results show that attention-based models generally provide stronger robustness under unstable Wi-Fi fingerprints. The MLP remains the strongest latency-first deployment option. The Set Transformer achieves the strongest joint classification (predicting building–floor pairs as a single label) and final deployment robustness score. The Standard Transformer performs best on multitask classification (predicting building and floor separately). For coordinate regression, the Set Transformer and MLP are nearly tied in mean error, with the Set Transformer achieving lower RMSE. Stability training improves retained performance under robustness tests, especially for classification tasks, although architecture-level tradeoffs remain. Overall, the project shows that indoor localization models should be evaluated not only by clean accuracy, but also by robustness and deployment cost.

aware evaluation is becoming the default expectation in this topic. Several indoor positioning papers emphasize that wireless fingerprints vary over time, across devices, and across environments, which can cause accuracy to degrade (e.g., [2]). Some indoor positioning works aim to reduce fingerprint ambiguity by introducing additional information such as trajectories (sequences) or richer physical-layer features (CSI), training sequence models (e.g., CNN–LSTM) and/or generating synthetic data[3, 4].

In this project, we study how model performance changes within unstable RSSI fingerprint conditions using a popular indoor WiFi fingerprints data, UJIIndoorLoc (UJI) [5]. Clean-split accuracy does not tell us which common changes/ degradations in RSSI fingerprints in real deployments cause localization error to increase. We treat fingerprint instability as a measurable set of factors (not as an unspecified noise term). We design controlled test degradations that vary (i) missing access-point readings, (ii) device-dependent RSSI bias, and (iii) random RSSI fluctuations, and we measure how localization error changes as these factors increase. Our goal is to identify which factor produces the largest performance drop and which model choices reduce the error increase, using matched test cases and a fixed tuning protocol so that performance comparisons across models are fair and comparable. In addition, we include the stability training to see if our trained models improve their robustness. We apply stability/consistency regularization by enforcing similar predictions under controlled input perturbations, following stability training methods studied in prior work [6, 7, 8].

1. Introduction/Background/Motivation

A recent survey summarizes the diversity of approaches (signal processing, ML/DL, and hybrid systems) and highlights recurring evaluation challenges such as data set differences and generalization limits when the evaluation is performed only on clean splits[1]. In recent studies, shift-

2. Approach

2.1 Dataset, Tasks, and Models

We use the UJIIndoorLoc dataset to study indoor localization under unstable Wi-Fi fingerprints. We evaluate two prediction settings: (1) building–floor classification and (2) 2D coordinate regression. Performance is measured by build-

ing accuracy, floor accuracy, joint building–floor accuracy, and Euclidean coordinate error, which are standard metrics in indoor localization research [1].

Under one common experimental framework, we compare five deep learning models: MLP, 1D CNN, bidirectional LSTM, standard Transformer, and Set Transformer [9]. The goal is not only to compare clean-test accuracy, but also to understand which model families remain reliable when Wi-Fi fingerprints become incomplete or corrupted.

2.2 Shared Pipeline and Model Representation

To keep the comparison consistent, all models use the same preprocessing pipeline, train/validation/test split, evaluation code, and degradation generator. Our preprocessing retains both normalized RSSI values and detection-mask values so that the models can distinguish weak signals from missing access points.

For the CNN and LSTM, we preserve the paired structure of the input by reorganizing the flat RSSI-plus-mask vector into per-access-point features to avoid underfitting issues. In the CNN, this representation is treated as a two-channel 1D signal, where each AP position contains its RSSI value and detection indicator. In the LSTM, the same paired per-AP structure is used as a sequence input. In addition, the LSTM supplements each AP token with a learned AP identity embedding so that the model can distinguish signal sources rather than treating access-point measurements as interchangeable.

For the transformer-based models, we convert the flat input into per-access-point tokens of shape $[520, 2]$, where each token contains the RSSI value and detection indicator for one AP to overcome underfitting issues. This preserves the local relationship between the two features associated with the same AP and provides a more natural input for attention over AP-level tokens. For the Set Transformer, we additionally preserve AP identity so that measurements from different access points are not treated as interchangeable.

2.3 Robustness Test and Stability Training

To evaluate robustness, we construct controlled robustness test sets using three perturbation types: AP dropout, RSSI bias, and Gaussian noise. AP dropout simulates incomplete scans, RSSI bias simulates device-dependent signal shift, and Gaussian noise simulates scan-to-scan fluctuation. These robustness tests let us measure how model performance changes as Wi-Fi fingerprints become less reliable.

We also evaluate a stability-training setting. In this setting, models are trained on a mixture of clean and perturbed samples while keeping the original labels unchanged. The training perturbations use the same mechanisms as the

degradation generator, but the evaluation suite includes broader held-out robustness scenarios so that improvement reflects robustness rather than memorization of a few hand-crafted cases.

2.4 Fair Comparison and Tuning Discipline

A main challenge in this project is that different architectures can appear better or worse because they use different preprocessing, parameter scales, or tuning effort. To reduce this problem, we enforce four rules. First, all models use the same data split, preprocessing, and evaluation metrics. Second, we keep model capacity in a similar range across architectures so that comparisons are not dominated by parameter count alone. Third, we start from a shared baseline training configuration and then tune only architecture-appropriate hyperparameters within a fixed model design. Fourth, all clean, degraded, and stability-trained evaluations are performed on the same test samples with fixed random seeds.

Our tuning policy remains conservative. We do not change the task definition, evaluation protocol, or degradation settings during tuning, and we select the final configuration for each model based on validation performance under the clean setting before running robustness experiments. This design makes the later degradation, stability, and latency comparisons more defensible because they compare reasonably tuned versions of each model under the same protocol.

We anticipated two main difficulties. The first was unfair comparison across architectures with different input formats and task heads. The second was that set-based processing might lose AP-specific meaning if AP identity were ignored. Early design choices confirmed that these issues mattered, so we standardized the experiment pipeline and preserved per-AP structure explicitly in the model inputs. Please refer to Appendix A for the standardization of architectures to account for input shape and AP identity. These decisions made the comparison more reliable and helped us interpret differences in robustness as model behavior rather than evaluation drift.

3. Experiments and Results

3.1 Setup and Training Scheme

We implemented five deep learning models—MLP, CNN, LSTM, standard Transformer, and Set Transformer—under a shared experimental framework. All models use the same preprocessing pipeline, train/validation/test split, evaluation metrics, and robustness-test generator. To support a fair comparison, we start from a shared baseline training configuration before applying model-specific tuning. All models were implemented in PyTorch. The Set Transformer follows the architecture of Lee et al. [9] and was

informed by the public reference implementation, with task-specific adaptations for Wi-Fi fingerprint localization. We also tracked trainable-parameter counts for every model and task; the full counts are reported in Appendix B.

Joint building–floor classification uses cross-entropy loss on the joint label, the building/floor multitask setting uses the sum of the building and floor cross-entropy losses, and coordinate regression uses mean squared error on the 2D target. These objectives are standard for the corresponding tasks and were kept consistent across model families.

We organized the experiments in five stages: (1) clean baseline training and evaluation, (2) hyperparameter tuning, (3) robustness testing under controlled perturbations, (4) stability training with augmented fingerprints, and (5) CPU inference latency benchmarking.

3.1.1 Clean Test

We first trained and evaluated all models on the original clean dataset to establish a common baseline. This stage serves as the reference point for all later robustness, stability-training, and latency comparisons. To keep the clean comparison consistent across architectures, we started from the same optimizer family, validation procedure, early-stopping policy, and logging setup, as summarized in Table 1. We report building accuracy, floor accuracy, joint building–floor accuracy, and Euclidean coordinate error.

Training Hyperparameter	Baseline Value
Optimizer	AdamW
Learning rate	2×10^{-3}
Weight decay	1×10^{-4}
Training batch size	256
Validation batch size	512
Shuffle training data	True
Pin memory	False
Maximum epochs	30
Early-stopping patience	5
Print frequency	Every 5 epochs
Minimum learning rate	1×10^{-5}
Enable logging	True

Table 1: Shared baseline training configuration.

3.1.2 Hyperparameter Tuning

After establishing the clean baseline, we tuned each architecture separately under the same preprocessing, split, and evaluation protocol. Our tuning policy was intentionally conservative: we kept the model family and task definition fixed, tuned only architecture-appropriate hyperparameters, and selected final configurations based on validation performance under the clean setting. This prevents later robustness comparisons from being dominated by weak default settings while avoiding tuning drift across models. The final tuned hyperparameters are reported in Appendix C (Table 8).

3.1.3 Robustness Test

We evaluated each model on controlled robustness tests designed to simulate realistic changes in Wi-Fi scan quality. These tests include AP dropout, RSSI bias, Gaussian noise, and their combinations. The dropout-only scenarios measure sensitivity to incomplete scans, the single-factor scenarios isolate systematic shift and random fluctuation, and the combined scenarios approximate more realistic mixed corruption. We append the evaluation settings in Appendix D.

3.1.4 Stability Training Test

To test whether robustness can be improved through training, we trained one version of each model on the original clean data and another on a mixed dataset containing both clean and perturbed samples. The training perturbations use the same three mechanisms as the degradation generator: AP dropout, RSSI bias, and Gaussian noise. However, the evaluation suite is broader than the training augmentation set, so improved performance reflects generalization to unseen robustness conditions rather than matching a small set of handcrafted perturbations.

3.1.5 Inference Latency Test

Finally, we measured per-sample batch-1 inference latency on CPU by running each model on a shared subset of 512 validation samples, repeating the sweep three times after warm-up, and summarizing performance with median and p95 latency. These measurements complement parameter-count and robustness comparisons by capturing deployment-oriented efficiency under the same preprocessing and software environment.

3.2 Result 1: Clean Test Comparison

Table 2 compares clean test-set performance across the five model families. Overall, the transformer-based models provide the strongest clean-set classification performance, and the Set Transformer is the strongest model for coordinate regression. The MLP remains a competitive lightweight baseline, but it is generally less accurate than the transformer-based models on the main classification tasks. CNN and LSTM performance is more mixed, reflecting the fact that these architectures impose stronger local or sequential assumptions on Wi-Fi fingerprints than the attention-based models.

For joint and multitask classification, the Standard Transformer achieves the best overall clean-test results, and the Set Transformer remains close and is consistently competitive. For coordinate regression, the Set Transformer achieves the lowest localization error among the compared models. These results suggest that attention-based architectures are a strong fit for Wi-Fi fingerprint localization under the shared evaluation protocol, with the Standard Trans-

Table 2: Clean test-set performance across model families.

(a) Joint Classification				
Model	Ep.	Score	Bldg.	Floor
MLP	22	0.9082	0.9955	0.9091
CNN	22	0.8326	0.9442	0.8425
LSTM	22	0.9478	0.9955	0.9478
Std. Trans.	12	0.9532	0.9982	0.9541
Set Trans.	35	0.9577	1.0000	0.9577
(b) Multitask Classification				
Model	Ep.	Score	Bldg.	Floor
MLP	11	0.9109	0.9973	0.9109
CNN	25	0.8074	0.9712	0.8182
LSTM	18	0.9415	0.9901	0.9505
Std. Trans.	36	0.9559	0.9991	0.9559
Set Trans.	22	0.9451	0.9964	0.9469
(c) Coordinate Regression				
Model	Ep.	Score	Mean m	RMSE m
MLP	55	-9.7672	9.7672	13.0668
CNN	56	-42.9523	42.9523	60.1932
LSTM	43	-11.3994	11.3994	15.5711
Std. Trans.	39	-10.3408	10.3408	13.9084
Set Trans.	44	-9.7647	9.7647	12.6781

Note. Ep. denotes the best validation epoch. Bldg. denotes building accuracy. Score is joint accuracy for classification tasks and negative mean Euclidean error in meters for coordinate regression. Mean m and RMSE m denote coordinate prediction error in meters. Higher classification scores are better, while lower coordinate errors are better.

former favoring classification and the Set Transformer favoring coordinate prediction.

We emphasize that the clean test is only one part of the comparison. The more important question for deployment is whether these performance advantages remain under robustness tests and whether they justify the added inference cost relative to simpler baselines such as MLP.

3.3 Result 2: Robustness Test Comparison

Figure 1 compares clean-trained models under dropout-only robustness tests. Retained performance is measured relative to each model’s clean-test performance, so values near 100% indicate that the model preserves most of its original performance under perturbation.

Across all three tasks, retained performance decreases as more detected AP readings are removed. CNN degrades most sharply, especially in coordinate regression, indicating that its local convolutional structure is not robust when AP observations become sparse. MLP is stronger than CNN but still loses substantial coordinate performance under high dropout. In contrast, the Standard Transformer and Set Transformer retain higher performance across most dropout rates, especially for joint and multitask classification. LSTM is also competitive after adding AP identity embeddings and attention pooling, but the transformer-

based models remain the most consistent under sparse fingerprints.

These results show that clean-test accuracy alone is not sufficient for evaluating indoor localization models. Models that perform similarly on clean data can behave very differently when Wi-Fi scans become incomplete. The attention-based models are more robust because they can use AP-level representations and aggregate information from the remaining observed signals more flexibly than CNN or MLP under dropout.

3.4 Result 3: Stability Training Comparison

Figure 2 shows the effect of stability training, where models are trained with augmented fingerprints generated using AP dropout, RSSI bias, and Gaussian noise. Compared with the clean-trained robustness test, stability training keeps retained performance closer to 100% across most dropout levels. The improvement is especially clear for joint and multitask classification, where MLP, LSTM, Standard Transformer, and Set Transformer all remain relatively stable under moderate and high dropout.

The special-scenario heatmaps in Figure 2b evaluate harder mixed perturbations that combine AP dropout, RSSI bias, and Gaussian noise. Under these combined robustness tests, stability-trained transformer models and LSTM retain high classification performance, while CNN remains substantially weaker. For coordinate regression, Set Transformer retains the strongest performance among the higher-capacity models, especially under the hardest combined scenario, `drop0.40_bias5_noise3`. Coordinate localization remains more sensitive than classification because small input perturbations can produce larger continuous-position errors.

Overall, stability training improves robustness, but it does not eliminate architecture-level differences. CNN remains the least reliable under unstable Wi-Fi fingerprints. MLP is attractive as a lightweight baseline, but attention-based models provide stronger robustness under severe perturbation. This supports the use of Standard Transformer for classification-focused localization and Set Transformer for coordinate-focused localization when robustness is prioritized.

3.5 Result 4: Inference Latency and Deployment Trade-off

Table 3 summarizes the inference latency protocol. We use CPU inference with batch size 1 because the intended deployment setting is single-scan Wi-Fi localization, where one fingerprint is processed at a time. Each model is evaluated on the same validation subset after a warm-up phase, and we report median and p95 latency to capture both typical and tail inference cost.

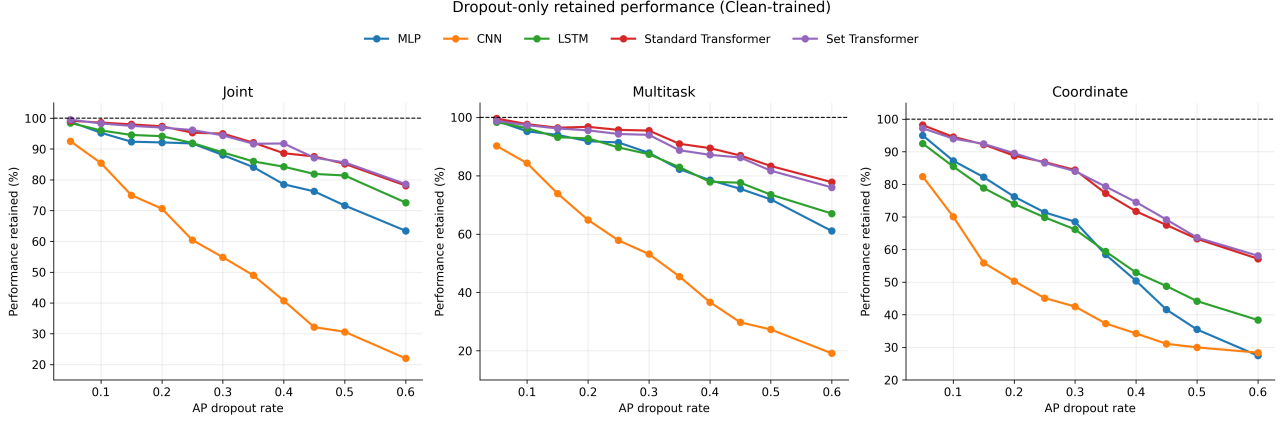
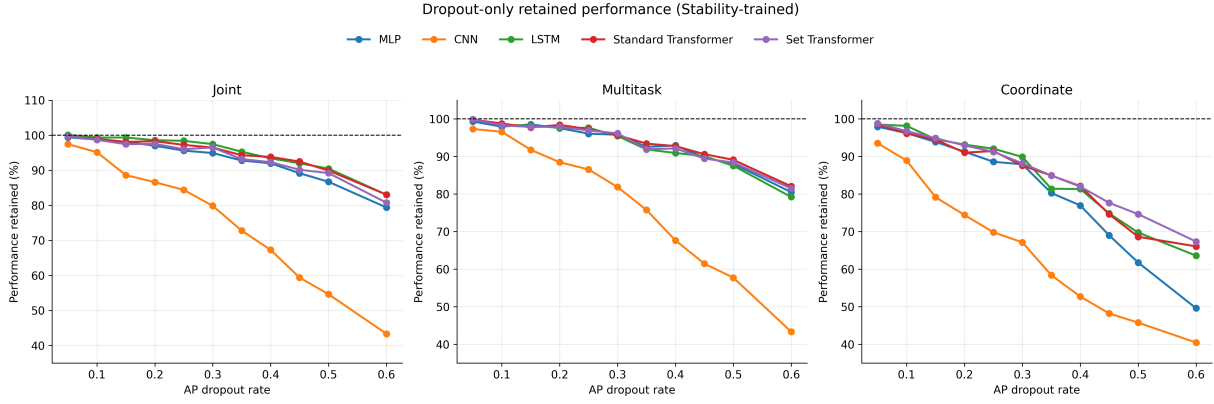
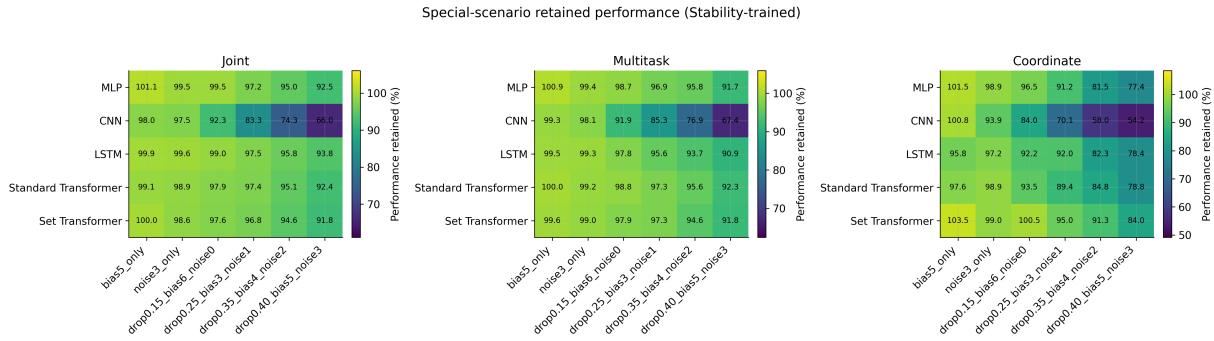


Figure 1: Dropout-only retained performance for clean-trained models. Retained performance is computed relative to each model’s clean-test performance; higher values indicate stronger robustness under AP dropout.



(a) Dropout-only retained performance after stability training.



(b) Special-scenario retained performance after stability training.

Figure 2: Stability-training results under robustness tests. Retained performance is reported relative to each model’s clean-test performance, where higher values indicate stronger robustness.

Table 4 summarizes the deployment tradeoff between robustness and CPU inference cost. The Set Transformer achieves the highest final deployment robustness score, followed closely by the Standard Transformer and LSTM. However, this robustness comes with substantially higher

CPU p95 latency. In contrast, MLP achieves a slightly lower robustness score but is orders of magnitude faster, making it the strongest latency-first deployment option. CNN is also fast, but its robustness score is much lower, especially under the worst special scenario.

Table 3: Inference latency evaluation configuration.

Setting	Value	Purpose
Device	CPU	Deployment-oriented comparison
Batch size	1	Single-scan inference
Samples	512	Shared validation subset
Warmup	50	Stabilize timing
Repeats	3	Reduce timing noise
Seed	42	Reproducible sampling
Percentile	95th	Tail-latency measure

Table 4: Deployment-oriented comparison using robustness score and CPU p95 latency. The final deployment score is the average of dropout robustness and worst-case special-scenario robustness. Higher robustness is better; lower latency is better.

Model	Dropout robust. avg.	Worst special robust.	Final deploy. score	CPU p95 lat. (ms)
Set Transformer	91.25	89.20	90.23	233.11
Std. Transformer	91.32	87.82	89.57	112.59
LSTM	91.27	87.72	89.49	152.99
MLP	89.28	87.18	88.23	0.17
CNN	72.63	62.55	67.59	1.07

This result shows that as shown in Appendix B, parameter count alone is not sufficient to explain deployment cost. Attention-based and sequence-based models introduce runtime overhead from token processing, sequence encoding, and pooling operations. Therefore, the practical deployment choice depends on the target constraint: Set Transformer is preferable when maximum robustness is prioritized, Standard Transformer offers a stronger robustness-latency compromise among attention-based models, and MLP is preferable when real-time CPU inference is the dominant requirement.

3.6 Project Success

We consider the project successful because it produced a controlled comparison across five model families—MLP, CNN, LSTM, Standard Transformer, and Set Transformer—using the same preprocessing pipeline, train/validation/test split, evaluation metrics, robustness-test scenarios, and CPU latency protocol. Success was measured not only by clean-test accuracy, but also by retained performance under AP dropout, RSSI bias, Gaussian noise, combined robustness tests, and batch-1 CPU inference latency.

A major lesson from the project was that fair comparison across different deep learning architectures is difficult. We controlled the dataset split, metrics, training protocol, random seeds, parameter scale, and robustness scenarios as much as possible. However, each model requires a different input representation to use its own inductive bias: MLP consumes the flat vector, CNN uses a two-channel AP signal,

LSTM uses an AP-level sequence with identity embeddings and attention pooling, and transformer models use AP-level tokens. This made the comparison stronger than using unrelated pipelines, but it also showed that fairness does not mean forcing every model into the same representation. We therefore treat the input transformations as a necessary design choice and a remaining limitation.

The results show that no single model is best under every constraint. Transformer-based models and the attention-pooled LSTM provide stronger robustness under unstable Wi-Fi fingerprints, while the MLP remains the strongest latency-first deployment option because of its extremely low CPU p95 latency. CNN was fast, but its robustness was substantially weaker, especially for coordinate regression. Thus, the project succeeded in identifying a practical deployment tradeoff rather than simply selecting the highest clean-test score.

A limitation is that our robustness tests are still simulated perturbations rather than new real-world scans from different buildings, devices, or time periods. Future work should evaluate the models on independently collected Wi-Fi fingerprints, test more realistic device-shift scenarios, and explore lighter attention-based architectures that preserve transformer-level robustness while reducing CPU latency.

We report the main comparison tables and figures in the paper. Full result CSVs, notebooks, and plotting scripts are included in the project repository [10].

4. Work Division

Student Name	Contributed Aspects	Details
Jason W. Park	Ideation, implementation, analysis, and documentation	Project ideation; implemented, trained, and fine-tuned the CNN and LSTM models; analyzed experimental results; and contributed to the proposal and final report. Project contact with TAs and other stakeholders. Implementation: notebooks/cnn and lstm related ipynb, src/models/cnn and lstm related python files.
Di Wu	Planning, implementation, analysis, and documentation	Planned and implemented shared libraries for the standard preprocessing pipeline, evaluation framework, and model training templates. Implemented the MLP baseline and the degradation and stability-training libraries. Contributed to the proposal and final report. Implementation: notebooks/mlp related and template ipynb for everybody, shared project utilities for preprocessing, training, and evaluation metrics under src, src/model/mlp related python files.
Kiyoshi Watanabe	Project management, implementation, analysis, and documentation	Managed project coordination; implemented, trained, and fine-tuned the standard Transformer and Set Transformer models; analyzed experimental results; and contributed to the proposal and developed the final report. Implementation: notebooks/transformer related ipynb, src/models/transformer related python files

Table 5: Contributions of team members.

This is the end of our main report body. Below are the references and appendices.

References

- [1] K. Roohi and A. R. Fekr, "A comparative analysis of indoor localization technologies," *Computer Networks*, vol. 270, p. 111527, 2025. 1, 2
- [2] Z. Tang, R. Gu, S. Li, K. S. Kim, and J. S. Smith, "Static vs. dynamic databases for indoor localization based on wi-fi fingerprinting: A discussion from a data perspective," *arXiv preprint arXiv:2402.12756*, 2024. 1
- [3] J. Li, J. Dong, J. G. Hwang, L. Rana, H. Ryu, and J. G. Park, "Deep learning-based wi-fi signal fingerprinting indoor positioning technology," in *Proceedings of the 37th International Technical Meeting of the Satellite Division of The Institute of Navigation (ION GNSS+ 2024)*, (Baltimore, Maryland), pp. 1354–1362, September 2024. 1
- [4] Z. Zhang, M. Lee, and S. Choi, "Deep-learning-based wi-fi indoor positioning system using continuous csi of trajectories," *Sensors*, vol. 21, no. 17, 2021. 1
- [5] J. Torres-Sospedra, R. Montoliu, A. Martínez-Usó, J. P. Avariento, T. J. Mauri, M. Benedito-Bordonau, and J. Huerta, "Ujiindoorloc: A new multi-building and multi-floor database for wlan fingerprint-based indoor localization problems," in *2014 International Conference on Indoor Positioning and Indoor Navigation (IPIN)*, pp. 261–270, IEEE, 2014. 1
- [6] S. Zheng, Y. Song, T. Leung, and I. Goodfellow, "Improving the robustness of deep neural networks via stability training," 2016. 1
- [7] J. Laermann, W. Samek, and N. Strodthoff, *Achieving Generalizable Robustness of Deep Neural Networks by Stability Training*, p. 360–373. Springer International Publishing, 2019. 1
- [8] T. Miyato, S. ichi Maeda, M. Koyama, and S. Ishii, "Virtual adversarial training: A regularization method for supervised and semi-supervised learning," 2018. 1
- [9] J. Lee, Y. Lee, J. Kim, A. R. Kosiorek, S. Choi, and Y. W. Teh, "Set transformer: A framework for attention-based permutation-invariant neural networks," in *International conference on machine learning*, pp. 3744–3753, PMLR, 2019. 2
- [10] K. Watanabe, "Cs7643 project: Indoor localization project." <https://github.com/iptracej-education/CS7643-Project-Final/>, 2026. GitHub repository. Accessed: 2026-04-29. 6

Appendix

A. Standardization of Architectures

Input Representation Transformations by Model

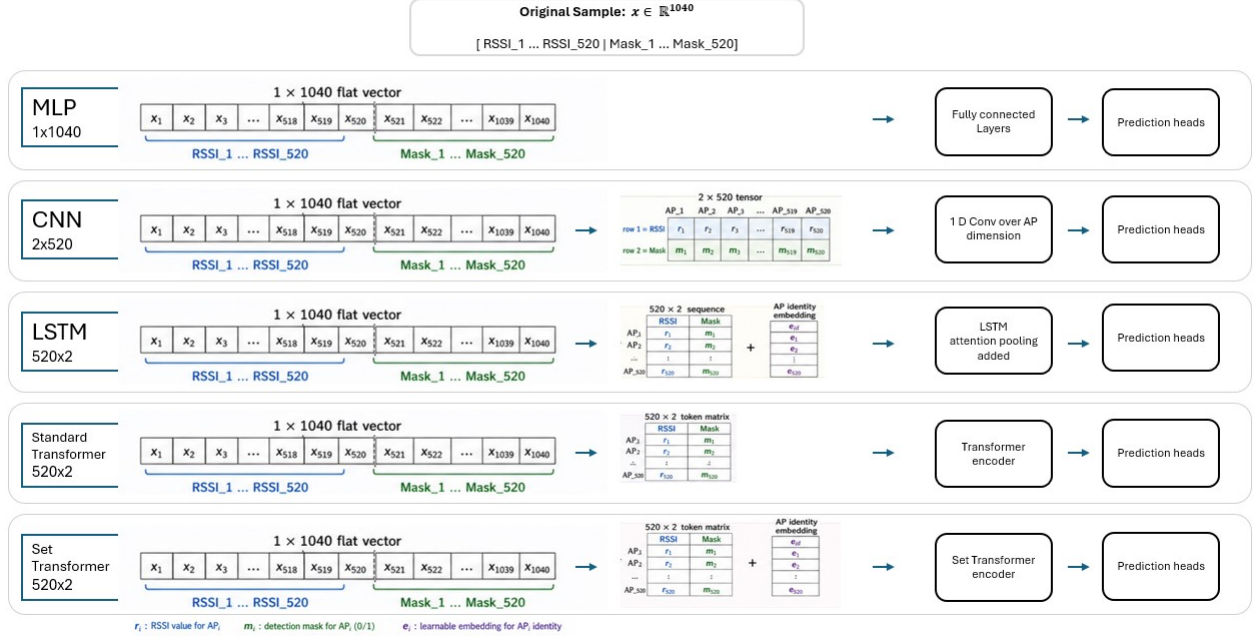


Figure 3: Standardization of architectures to account for input shape and AP identity.

B. Trainable Parameter Counts

Table 6: Number of trainable parameters for each model and task.

Model	Task	Parameters	Millions
CNN	Coordinate	1,582,082	1.582
CNN	Joint	1,584,909	1.585
CNN	Multitask	1,583,624	1.584
LSTM	Coordinate	1,639,203	1.639
LSTM	Joint	1,642,030	1.642
LSTM	Multitask	1,640,745	1.641
MLP	Coordinate	1,726,210	1.726
MLP	Joint	1,729,037	1.729
MLP	Multitask	1,727,752	1.728
Standard Transformer	Coordinate	349,442	0.349
Standard Transformer	Joint	350,861	0.351
Standard Transformer	Multitask	366,728	0.367
Set Transformer	Coordinate	481,794	0.482
Set Transformer	Joint	483,213	0.483
Set Transformer	Multitask	499,080	0.499

C. Final Hyperparameter Tuning Protocol and Selected Configurations

We used a bounded hyperparameter-tuning protocol to keep tuning effort comparable across model families. The model architectures were fixed before tuning, and we tuned only training and regularization parameters such as learning rate, weight decay, maximum epochs, early-stopping patience, dropout where applicable, and gradient clipping where configured. Fi-

nal configurations were selected by validation performance under the clean setting before running robustness and latency experiments.

This protocol is not identical tuning across all architectures. Instead, it applies equal tuning discipline: all models were selected from a small predefined search space, using the same data split, validation metrics, and final evaluation protocol. This design reduces the risk that one architecture benefits from substantially more tuning effort than another.

Table 7: Controlled hyperparameter tuning grid. The same bounded training grid was used to select final configurations, while gradient clipping was enabled only for model families where it was configured in the final training run.

Grid	LR	Weight Decay	Max Epochs	Patience
Grid 1	2.0×10^{-3}	1.0×10^{-4}	30	5
Grid 2	1.0×10^{-3}	1.0×10^{-4}	50	10
Grid 3	1.0×10^{-3}	5.0×10^{-4}	50	10
Grid 4	5.0×10^{-4}	1.0×10^{-4}	60	12
Grid 5	5.0×10^{-4}	5.0×10^{-4}	80	15

Note. The LSTM used the same bounded tuning discipline but included gradient clipping because recurrent models are more sensitive to optimization instability. Transformer-family models also used gradient clipping in the final selected runs. MLP and CNN did not use gradient clipping.

Table 8: Selected tuned configurations from the controlled grid used in the final experiments.

Model	Task	LR	Weight Decay	Max Epochs	Patience	Grad Clip	Dropout	d_{model}	Heads	Depth / FFN / Seeds
CNN	Coordinate	5.0×10^{-4}	1.0×10^{-4}	60	12	–	–	–	–	–
CNN	Joint	1.0×10^{-3}	5.0×10^{-4}	50	10	–	–	–	–	–
CNN	Multitask	1.0×10^{-3}	1.0×10^{-4}	50	10	–	–	–	–	–
LSTM	Coordinate	1.0×10^{-3}	1.0×10^{-4}	50	10	1.0	–	–	–	–
LSTM	Joint	5.0×10^{-4}	5.0×10^{-4}	80	15	1.0	–	–	–	–
LSTM	Multitask	5.0×10^{-4}	1.0×10^{-4}	60	12	1.0	–	–	–	–
MLP	Coordinate	5.0×10^{-4}	5.0×10^{-4}	80	15	–	–	–	–	–
MLP	Joint	1.0×10^{-3}	5.0×10^{-4}	50	10	–	–	–	–	–
MLP	Multitask	1.0×10^{-3}	5.0×10^{-4}	50	10	–	–	–	–	–
Standard Transformer	Coordinate	1.0×10^{-3}	1.0×10^{-4}	50	10	1.0	0.1	128	4	2/256/ – –
Standard Transformer	Joint	1.0×10^{-3}	5.0×10^{-4}	50	10	1.0	0.1	128	4	2/256/ – –
Standard Transformer	Multitask	1.0×10^{-3}	1.0×10^{-4}	50	10	1.0	0.1	128	4	2/256/ – –
Set Transformer	Coordinate	1.0×10^{-3}	5.0×10^{-4}	50	10	1.0	0.1	128	4	2/256/1
Set Transformer	Joint	5.0×10^{-4}	5.0×10^{-4}	80	15	1.0	0.1	128	4	2/256/1
Set Transformer	Multitask	5.0×10^{-4}	5.0×10^{-4}	80	15	1.0	0.1	128	4	2/256/1

D. Evaluation Settings

Scenario Group	Scenario	Description
Clean	clean	Original test set without modification.
Dropout-only	dropout_0.05	Remove 5% of detected AP readings.
	dropout_0.10	Remove 10% of detected AP readings.
	dropout_0.15	Remove 15% of detected AP readings.
	dropout_0.20	Remove 20% of detected AP readings.
	dropout_0.25	Remove 25% of detected AP readings.
	dropout_0.30	Remove 30% of detected AP readings.
	dropout_0.35	Remove 35% of detected AP readings.
	dropout_0.40	Remove 40% of detected AP readings.
	dropout_0.45	Remove 45% of detected AP readings.
	dropout_0.50	Remove 50% of detected AP readings.
	dropout_0.60	Remove 60% of detected AP readings.
Combined degradation	drop0.15.bias6.noise0	Dropout 0.15, RSSI bias +6 dB, noise std 0.
	drop0.25.bias3.noise1	Dropout 0.25, RSSI bias +3 dB, noise std 1.
	drop0.35.bias4.noise2	Dropout 0.35, RSSI bias +4 dB, noise std 2.
	drop0.40.bias5.noise3	Dropout 0.40, RSSI bias +5 dB, noise std 3.
Single-factor shift	bias5_only	Add a constant +5 dB offset to detected RSSI values.
	noise3_only	Add Gaussian noise with standard deviation 3.

Table 9: Degradation test scenarios used to evaluate model robustness.